

Module 6: Cybersecurity Fundamentals

Day 5 - SIEM Fundamentals



Module 6 — Course Overview

Day	Date	Topic
✓ Day 1	Mar 18	Threat Intelligence & Frameworks
✓ Day 2	Mar 25	URL Analysis & Vulnerability Fundamentals
✓ Day 3	Apr 15	Phishing & Email Security
✓ Day 4	Apr 22	Password Security & Network Traffic Analysis
▶ Day 5	Apr 29	Malware Analysis & SIEM
Day 6	May 6	Incident Response Reporting
Day 7	May 13	Capstone — Simulated Incident Investigation

What We Learned So Far — Days 1–4

- Day 1 — Threat Intelligence: Kill Chain, ATT&CK, OSINT, STIX/TAXII, ISACs
- Day 2 — URL & Vulnerabilities: Nmap, Nessus, CVEs, OWASP Top 10, Metasploit
- Day 3 — Phishing & Email Security: BEC, quishing, vishing, SPF/DKIM/DMARC
- Day 4 — Password Security & Network Traffic:
 - Password hashing, Hashcat, John the Ripper, PtH, Kerberoasting
 - Network traffic analysis, Wireshark, C2 detection, DNS tunneling, ARP spoofing

What is Malware?

- **Malware:** Malicious software designed to harm, exploit, or compromise systems
- **Common types:**
 - **Virus:** Self-replicating, attaches to files
 - **Worm:** Self-propagating across networks
 - **Trojan:** Disguised as legitimate software
 - **Ransomware:** Encrypts data for payment
 - **Spyware:** Steals information
 - **Rootkit:** Hides presence, maintains access

Malware Types — Know the Difference

Type	How It Spreads	Key Behavior	Famous Example
Virus	Attaches to files; user shares	Needs a host file to survive	ILOVEYOU (2000)
Worm	Network exploits; fully automatic	No user action needed	WannaCry (2017)
Trojan	Disguised as legit software	User installs it; opens backdoor	Emotet
Ransomware	Phishing, exploits, worms	Encrypts files; demands payment	LockBit, REvil
Spyware	Bundled / zero-click exploits	Records keystrokes, screens, creds	Pegasus (NSO)
Rootkit	Post-compromise installation	Hides in OS kernel; invisible	Stuxnet (2010)
RAT	Phishing, trojans, exploits	Full remote control of machine	Cobalt Strike

Malware can be multiple types simultaneously: WannaCry = Worm + Ransomware | Stuxnet = Worm + Rootkit | Emotet = Trojan + Loader

Malware Analysis Goals

- **Why analyze malware?**
- **Identification:** What is this malware?
- **Capabilities:** What can it do?
- **Indicators:** How do we detect it?
- **Attribution:** Who created it?
- **Remediation:** How do we remove it?
- **Analysis informs defense**

Analysis Types

Type	Method	Skill Level	Time
Automated	Sandbox submission	Low	Minutes
Static	Examine without running	Medium	Hours
Dynamic	Execute and observe	Medium-High	Hours
Code Analysis	Reverse engineering	High	Days

Safe Analysis Environment

- **Safety requirements:**
 - Isolated virtual machines
 - No network connectivity (or monitored fake network)
 - Snapshots before execution
 - Separate from production network
 - Host-based protections
- **Never analyze malware on production systems!**

Automated Sandbox Analysis

- **Online sandboxes:**
 - **Any.run:** Interactive analysis
 - **Hybrid Analysis:** Detailed reports
 - **VirusTotal:** Multi-engine scanning
 - **Joe Sandbox:** Comprehensive analysis
 - **Cuckoo Sandbox:** Self-hosted option (outdated!)
- **Submit sample → Receive report**

Sandbox Report Analysis

- **Report sections:**
 - **File information:** Hashes, type, size
 - **Detection:** AV engine results
 - **Network activity:** DNS, HTTP, connections
 - **File activity:** Created, modified, deleted files
 - **Registry activity:** Keys modified
 - **Process activity:** Spawned processes
 - **Extracted IOCs:** IPs, domains, hashes

Static Analysis Basics

- **Static analysis examines files without execution:**
 - **Techniques:**
 - File type identification
 - Hash calculation
 - String extraction
 - Metadata analysis
 - Packer detection
 - Import/export analysis
 - **Tools:** file, strings, exiftool, PEStudio, DIE

String Analysis

- **Strings reveal:**
 - URLs and IP addresses
 - File paths
 - Registry keys
 - Error messages
 - Commands
 - Encryption keys (sometimes)
- **Linux/Windows:**

```
strings malware.exe | grep -i "http"  
strings -a -n 6 malware.exe
```


PE File Analysis

- **PE (Portable Executable):** Windows executable format
- **Analysis points:**
 - Headers (compilation time, entry point)
 - Sections (.text, .data, .rsrc)
 - Imports (DLLs and functions used)
 - Exports (functions provided)
 - Resources (embedded files)
- **Tools:** PEStudio, PE-bear, pestudio

Identifying Suspicious Imports

Function	Purpose
VirtualAlloc	Memory allocation (shellcode)
CreateRemoteThread	Code injection
WriteProcessMemory	Process manipulation
RegSetValueEx	Registry modification
InternetOpen	Network communication
CryptEncrypt	Encryption (ransomware)

Packer Detection

- **Packers:** Compress/encrypt malware to avoid detection
- **Indicators of packing:**
 - High entropy (randomness)
 - Few readable strings
 - Small import table
 - Unusual section names
 - Known packer signatures
- **Tools:** Detect It Easy (DIE), PEiD, Exeinfo PE

Dynamic Analysis Basics

- **Dynamic analysis executes malware to observe behavior:**
- **What to monitor:**
 - Process creation
 - File system changes
 - Registry modifications
 - Network connections
 - API calls
- **Tools:** Process Monitor, Procmon, Wireshark, API Monitor

Process Monitor (ProcMon)

- **Process Monitor:** Windows real-time monitoring tool
- **Captures:**
 - File system operations
 - Registry operations
 - Network activity
 - Process/thread activity
 - Filters:** Focus on malware process, exclude noise
- **Essential for Windows malware analysis**

Document Malware Analysis

- **Malicious documents:**
 - Office files with macros (DOC, DOCX, XLS)
 - PDFs with JavaScript or embedded objects
 - RTF with exploits
- **Analysis approach:**
 - 1.Extract macros/scripts
 - 2.Deobfuscate code
 - 3.Identify payload delivery
 - 4.Execute in sandbox if needed
- **Tools:** oletools, olevba, pdf-parser

Analyzing Office Macros

- **olevba:** Extract and analyze VBA macros
- **Look for:** Auto-execution, shell commands, downloads

```
olevba malicious.doc
```

```
# Output shows:  
# - AutoOpen/AutoExec (automatic execution)  
# - Suspicious keywords (Shell, PowerShell, http)  
# - Obfuscation indicators  
# - Extracted macro code
```

Malware Analysis Workflow

- **Recommended workflow:**
 - 1. Gather info:** Hashes, context, source
 - 2. Automated analysis:** Submit to sandbox
 - 3. Static analysis:** Strings, PE analysis, scripts
 - 4. Dynamic analysis:** Execute and monitor
 - 5. Document findings:** IOCs, capabilities, recommendations
 - 6. Share intelligence:** Report to team/community

Why Memory Forensics?

- **Memory contains:**
 - Running processes (including hidden)
 - Network connections
 - Encryption keys
 - Decrypted malware
 - User credentials
 - Command history
- **"Memory doesn't lie"**

Memory Acquisition

- **Acquisition tools:**
 - **FTK Imager:** Free, reliable
 - **WinPMEM:** Open source
 - **Magnet RAM Capture:** Free
 - **LiME:** Linux Memory Extractor
- **Considerations:**
 - Acquisition changes memory
 - Time-sensitive (volatile)
 - Large files (GB)
 - Live system required

Volatility Framework

- **Volatility:** Premier memory forensics framework
- **Capabilities:**
 - Process listing (including hidden)
 - Network connections
 - Loaded DLLs
 - Registry extraction
 - Malware detection
- **Versions:** Volatility 2 (Python 2), Volatility 3 (Python 3)

Volatility Profile Selection

- Profile = Memory structure definition
- Volatility 2:
- Volatility 3:
- Volatility 3 auto-detects profiles

```
volatility -f memory.dmp imageinfo  
volatility -f memory.dmp --profile=Win10x64_19041 pslist  
vol.py -f memory.dmp windows.info  
vol.py -f memory.dmp windows.pslist
```

Essential Volatility Plugins

Plugin	Purpose
pslist/pstree	List processes
netscan	Network connections
malfind	Find injected code
dlllist	Loaded DLLs
handles	Open handles
cmdline	Command line arguments
hashdump	Extract password hashes

Finding Hidden Processes

- **Detecting hidden processes:**

```
# Normal process list (can be manipulated)
vol.py -f mem.dmp windows.pslist

# Scan for process structures (harder to hide)
vol.py -f mem.dmp windows.psscan
```

- **Compare results:** Processes in psscan but not pslist are hidden

Detecting Code Injection

- **malfind plugin:** Detects suspicious memory regions

```
vol.py -f mem.dmp windows.malfind
```

- **Indicators:**
 - Executable memory not backed by file
 - PAGE_EXECUTE_READWRITE permissions
 - Suspicious content (shellcode patterns)
- **Dumps suspicious regions for analysis**

Network Connection Analysis

- **Network artifacts in memory:**
- **Shows:**
 - Local and remote addresses
 - Ports and protocols
 - Connection state
 - Associated process
- **Find C2 connections from memory**

```
vol.py -f mem.dmp windows.netscan
```


Memory Forensics Workflow

- **Analysis workflow:**
 - 1.Acquire memory (before shutdown!)**
 - 2.Identify profile/OS**
 - 3.List processes (pslist/psscan comparison)**
 - 4.Check network connections (netscan)**
 - 5.Detect injection (malfind)**
 - 6.Extract artifacts (dump suspicious processes)**
 - 7.Document and correlate findings**

What is a SIEM?

- **SIEM = Security Information and Event Management**
- **Core functions:**
 - Log collection and aggregation
 - Normalization and parsing
 - Correlation and analysis
 - Alerting and notification
 - Investigation and search
 - Reporting and compliance

SIEM Data Flow

- **Log flow:**

Sources → Collection → Parsing → Storage → Analysis → Action

Endpoints	]	
Servers	]	→ Forwarders → SIEM → Index → Rules → Alerts
Network	]	↓
Applications]		Search/Dashboards

Log Sources

Source	Value
Windows Events	Authentication, process, PowerShell
Linux Syslog	Auth, system, application logs
Firewall	Network allows/denies
Proxy/Web	URL access, downloads
DNS	Query logs for detection
EDR	Endpoint detection data
Cloud	AWS/Azure/GCP audit logs

Windows Event Forwarding

- **Windows Event Forwarding (WEF):**
 - Native Windows log collection
 - Centralized subscription model
 - No agent required
 - Scalable to large environments
- **Key events to collect:**
 - Security (4624, 4625, 4688, 4672)
 - PowerShell (4103, 4104)
 - Sysmon (if installed)

Sysmon — System Monitor

- Free Microsoft Sysinternals tool — deep Windows activity logging
- Key Event IDs:
 - Event 1 — Process Create (full command line + parent process + hashes)
 - Event 3 — Network Connection (process name, src/dst IP, port)
 - Event 7 — Image Load (DLLs loaded — detect DLL hijacking)
 - Event 11 — File Create
 - Event 22 — DNS Query (detect DNS-based C2)
- Install: `sysmon64.exe -accepteula -i sysmonconfig.xml`
- Recommended config: SwiftOnSecurity or Olaf Hartong (GitHub)
- Output → Event Log: Applications and Services → Microsoft → Windows → Sysmon/Operational

Critical Windows Event IDs

- Authentication:
 - 4624 — Successful logon | 4625 — Failed logon (brute force)
 - 4648 — Logon with explicit credentials | 4672 — Special privileges assigned
 - 4776 — NTLM authentication attempt
- Account Management:
 - 4720 — Account created | 4726 — Account deleted
 - 4732/4728 — Added to security group / Domain Admins
- Process / Execution:
 - 4688 — Process created (enable command-line logging in audit policy!)
 - 4104 — PowerShell script block logging
- Services / Persistence:
 - 7045 — New service installed | 4698 — Scheduled task created



CYBERLABS SOAR — Security Orchestration, Automation and Response

- Extends SIEM: automates repetitive analyst tasks using playbooks
- Example playbook — Phishing Alert:
 - 1. Parse email → extract URLs, attachments, sender
 - 2. Submit to VirusTotal / sandbox automatically
 - 3. If malicious: block IP/domain at firewall, quarantine email
 - 4. Notify affected user | 5. Create incident ticket
- Tools: Splunk SOAR (Phantom), Palo Alto XSOAR, Microsoft Sentinel Logic Apps
- SOAR shifts analyst role: alert handler → playbook builder + exception reviewer
- Risk: automate only well-understood, high-confidence detections

Sigma Rules — Platform-Agnostic Detection

- Sigma: open standard for writing SIEM detection rules
- Write once → convert to any SIEM: Splunk, Elastic, QRadar, Sentinel
- Like Snort (network) or YARA (files) — but for log-based detection
- Rule structure: title | logsource (product/category) | detection | level
- Example: detect Mimikatz by process name + command line
- Repository: github.com/SigmaHQ/sigma — 5,000+ community rules
- Converter: pySigma (python) → generates native SIEM query
- Benefit: vendor-neutral — share detections across the entire community

Syslog Collection

- **Syslog:** Standard Unix/Linux logging protocol
- **Components:**
- **Facility:** Log source category
- **Severity:** Message importance
- **Message:** Log content
- **Collection:**
- **UDP (514) or TCP (514/6514)**

```
# rsyslog forwarding  
*. * @@siem.company.com:514
```


Log Parsing and Normalization

- Parsing transforms raw logs:

- Raw:

```
Jan 15 10:23:45 server sshd[1234]: Failed password for  
admin from 192.168.1.100
```

- Parsed:

```
{  
  "timestamp": "2024-01-15T10:23:45",  
  "host": "server",  
  "service": "sshd",  
  "event": "failed_login",  
  "user": "admin",  
  "source_ip": "192.168.1.100"  
}
```

Detection Rules and Correlation

- **Detection approaches:**
- **Single event:** One event triggers alert
- **Threshold:** Count exceeds limit
- **Correlation:** Multiple events combine
- **Sequence:** Events in specific order
- **Example correlation:**

```
IF failed_login count > 5 within 5 minutes  
AND source_ip same  
THEN alert "Brute Force Attempt"
```

Alert Management

- **Alert lifecycle:**
 1. **Triggered:** Rule matches, alert created
 2. **Triaged:** Analyst reviews priority
 3. **Investigated:** Deep dive into evidence
 4. **Escalated:** If confirmed, incident created
 5. **Closed:** False positive or resolved
- **Key metrics:** Alert volume, false positive rate, time to triage

SIEM Platforms

- **Common SIEM platforms:**
 - **Splunk:** Market leader, powerful, expensive
 - **Elastic Security:** Open source core, scalable
 - **Microsoft Sentinel:** Cloud-native, Azure integration
 - **IBM QRadar:** Enterprise-focused
 - **LogRhythm:** Mid-market option
 - **Wazuh:** Open source, endpoint-focused

Splunk Basics

- **Splunk components:**
 - **Search Processing Language (SPL)**
 - **Indexes:** Data storage
 - **Sourcetypes:** Log format definitions
 - **Apps:** Extended functionality
 - **Dashboards:** Visualization
 - **Basic search:**

```
index=security sourcetype=WinEventLog EventCode=4625
```


Elastic Stack (ELK)

- **Elastic Stack components:**
 - **Elasticsearch:** Search and storage engine
 - **Logstash:** Log processing pipeline
 - **Kibana:** Visualization and search UI
 - **Beats:** Lightweight data shippers
 - **Query language:** KQL (Kibana Query Language)
- **Open source + commercial features**

Day 5 Summary

- Malware analysis: automated sandbox → static (strings, PE imports) → dynamic (ProcMon)
- Memory forensics: Volatility plugins — pslist, netscan, malfind, dlllist
- SIEM: centralised log collection, normalisation, correlation rules, alerting
- Sysmon: deep Windows logging — process create, network connect, DNS query (Events 1/3/22)
- Critical Windows Event IDs: 4624/4625 (logon), 4688 (process), 4104 (PowerShell), 7045 (service)
- SOAR: automates alert playbooks — block, notify, ticket, enrich
- Sigma rules: write once, convert to any SIEM platform

© CyberLabs Technologies

Thank You

